MODUL 7

Binary Tree dan Traversal

Modul Praktikum Struktur Data • C++17

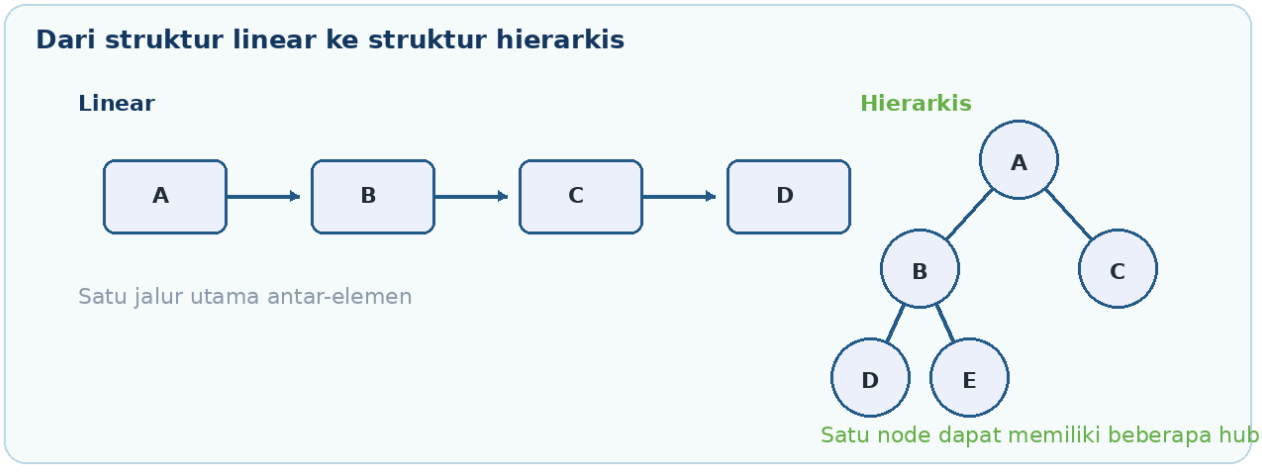
Program Studi Informatika • Praktikum Struktur Data • C++17

1. Tujuan Praktikum

1. menjelaskan perbedaan struktur linear dan struktur hierarkis;
2. mengenali root, parent, child, sibling, leaf, subtree, level, left child, dan right child;
3. merepresentasikan binary tree menggunakan node dengan pointer left dan right serta pointer root;
4. menjelaskan sifat rekursif tree dan kondisi dasar rekursi;
5. melakukan traversal preorder, inorder, dan postorder;
6. memprediksi urutan traversal sebelum menjalankan program;
7. menghitung jumlah node secara rekursif;
8. menangani tree kosong, satu node, dan bentuk tree tidak seimbang; dan
9. membebaskan seluruh node tree dengan benar.

2. Dasar Binary Tree

Struktur data yang telah dipelajari sebelumnya, seperti linked list, stack, dan queue, terutama membentuk hubungan linear. Tree digunakan ketika data mempunyai hubungan hierarkis: sebuah node dapat mempunyai hubungan ke beberapa node di bawahnya.



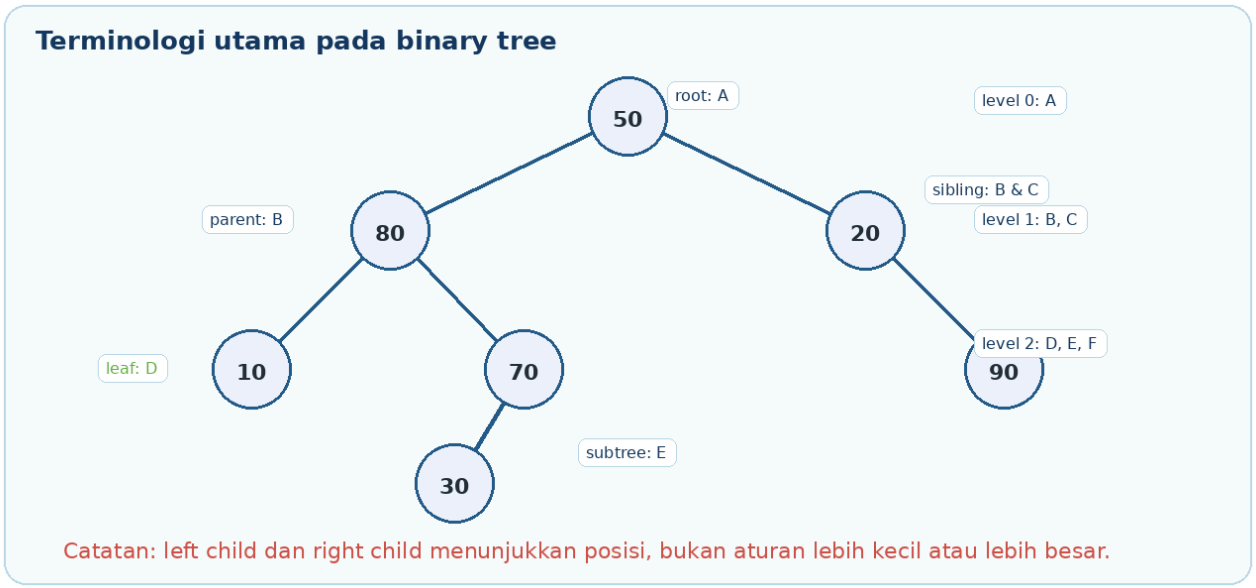
Gambar 1. Perbedaan struktur linear dan struktur hierarkis.

2.1 Terminologi tree

Tree terdiri atas node yang saling terhubung secara hierarkis. Istilah yang digunakan pada modul ini adalah:

• root: node paling atas yang menjadi titik awal tree;

- parent: node yang mempunyai child;
- child: node yang berada langsung di bawah sebuah parent;
- sibling: node-node yang mempunyai parent yang sama;
- leaf: node yang tidak mempunyai child;
- subtree: bagian tree yang berakar pada suatu node;
- level: posisi node terhadap root; dan
- left child serta right child: posisi child kiri dan kanan pada binary tree.



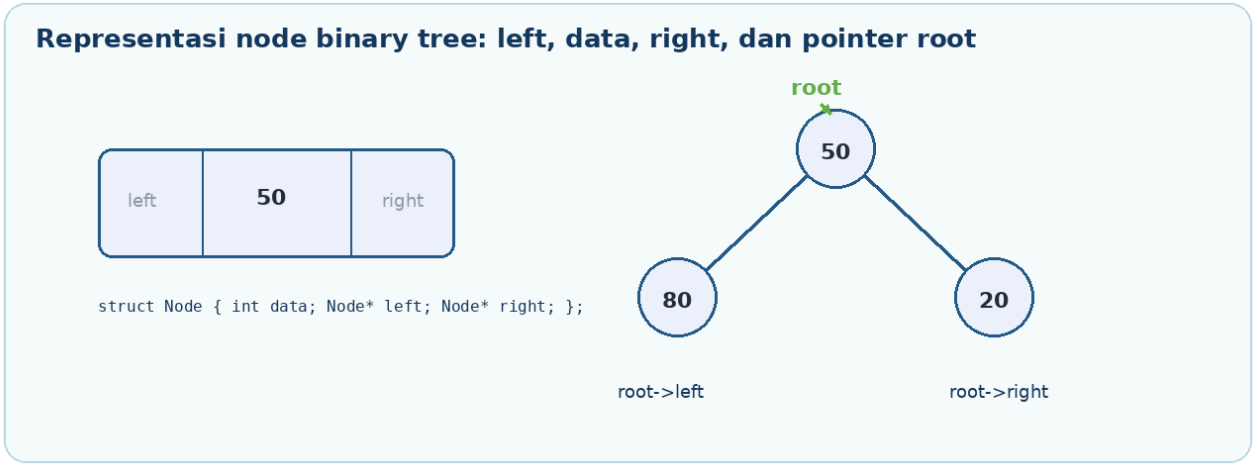
Gambar 2. Terminologi utama pada binary tree.

2.2 Binary tree

Binary tree adalah tree yang setiap nodenya mempunyai paling banyak dua child. Kedua hubungan tersebut disebut left child dan right child. Pada binary tree biasa, posisi left dan right tidak menyatakan bahwa nilai kiri harus lebih kecil atau nilai kanan harus lebih besar.

```
struct Node {
    int data;
    Node* left;
    Node* right;
};

Node* root = nullptr;
```



Gambar 3. Representasi node binary tree dan pointer root.

2.3 Keadaan binary tree

Pada tree kosong, root bernilai nullptr.

```
root == nullptr
```

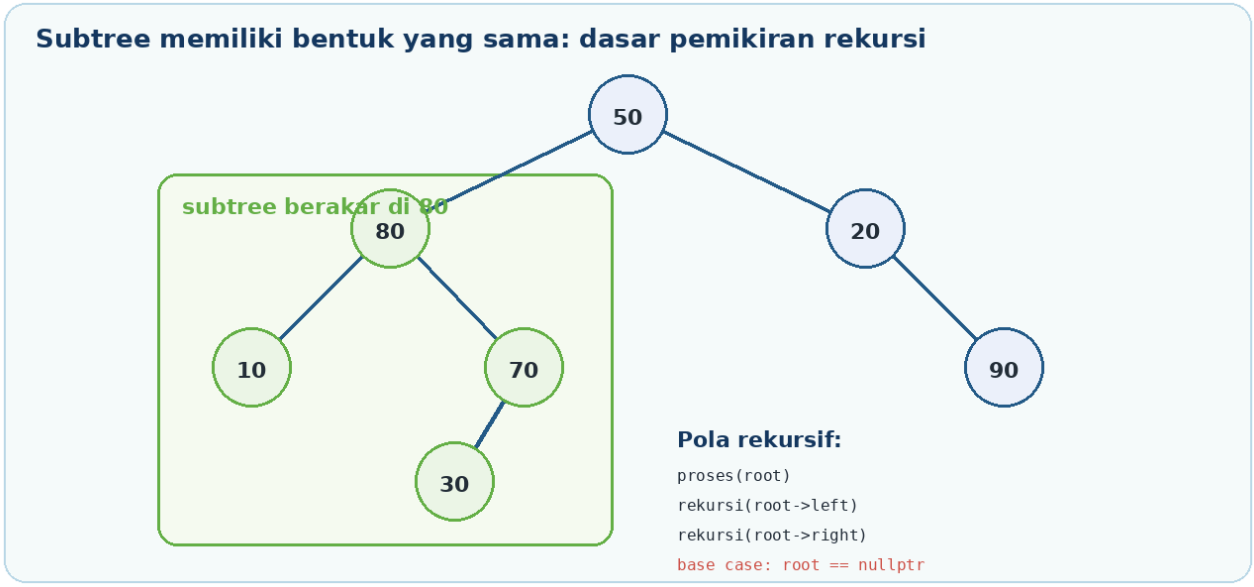
Pada tree satu node, root menunjuk satu node yang kedua child-nya bernilai nullptr.

```
root != nullptr
root->left == nullptr
root->right == nullptr
```

Node tersebut sekaligus merupakan root dan leaf.

3. Rekursi pada Tree

Setiap child dapat menjadi root sebuah subtree. Karena subtree mempunyai bentuk struktur yang sama dengan tree keseluruhan, operasi pada tree secara alami dapat dinyatakan secara rekursif.



Gambar 4. Subtree mempunyai struktur yang sama dan mendukung pola rekursif.

Kondisi dasar rekursi adalah subtree kosong:

```
if (root == nullptr)
    return;
```

Setelah kondisi dasar diperiksa, fungsi dapat memproses node saat ini dan memanggil dirinya sendiri untuk subtree kiri dan subtree kanan. Posisi proses terhadap dua pemanggilan rekursif menentukan jenis traversal.

4. Traversal Binary Tree

Traversal adalah proses mengunjungi setiap node tree menurut urutan tertentu. Modul ini menggunakan tiga traversal depth-first: preorder, inorder, dan postorder.

4.1 Preorder

Preorder menggunakan urutan Root - Left - Right. Node diproses sebelum kedua subtree.

```
void preorder(const Node* root) {
    if (root == nullptr)
        return;

    std::cout << root->data << ' ';
    preorder(root->left);
    preorder(root->right);
}
```

4.2 Inorder

Inorder menggunakan urutan Left - Root - Right. Node diproses setelah subtree kiri dan sebelum subtree kanan.

```
void inorder(const Node* root) {
    if (root == nullptr)
```

```
        return;

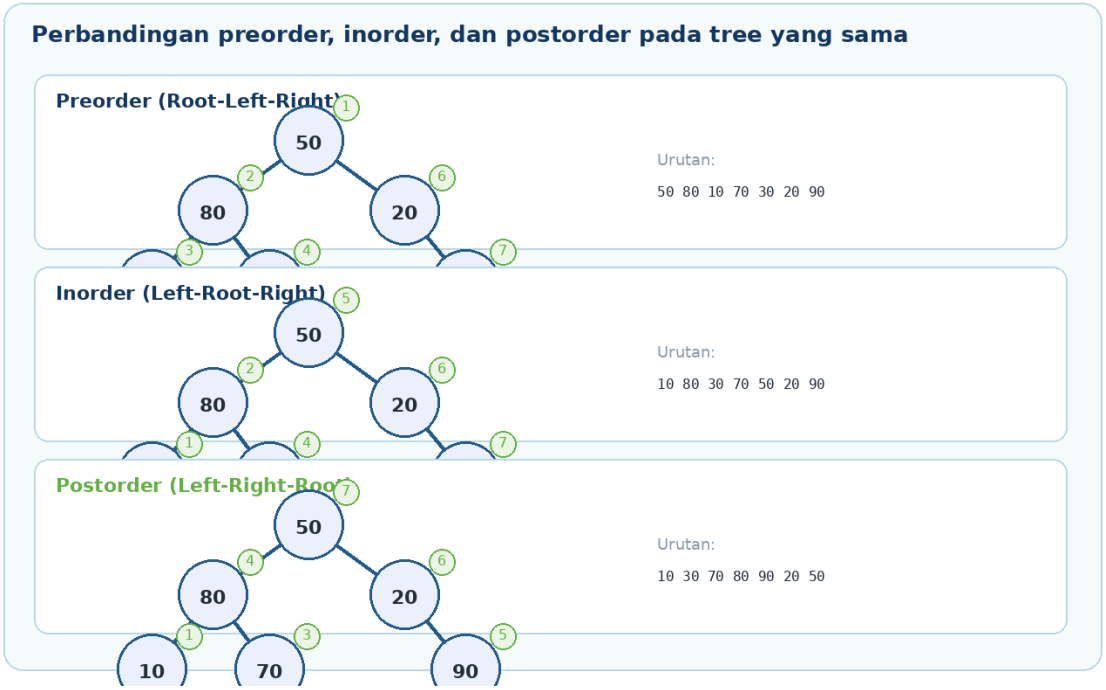
    inorder(root->left);
    std::cout << root->data << ' ';
    inorder(root->right);
}
```

4.3 Postorder

Postorder menggunakan urutan Left - Right - Root. Node diproses setelah kedua subtree.

```
void postorder(const Node* root) {
    if (root == nullptr)
        return;

    postorder(root->left);
    postorder(root->right);
    std::cout << root->data << ' ';
}
```



Gambar 5. Perbandingan preorder, inorder, dan postorder pada tree yang sama.

4.4 Prediksi sebelum menjalankan program

Sebelum menjalankan fungsi traversal, tuliskan terlebih dahulu urutan node berdasarkan gambar tree. Setelah itu jalankan program dan bandingkan output dengan prediksi. Langkah ini membantu membedakan kapan root diproses pada ketiga traversal.

5. Operasi Rekursif Tambahan

5.1 Menghitung jumlah node

Jumlah node pada sebuah tree adalah satu node saat ini ditambah jumlah node pada subtree kiri dan subtree kanan.

```
int jumlahNode(const Node* root) {
    if (root == nullptr)
        return 0;

    return 1
         + jumlahNode(root->left)
         + jumlahNode(root->right);
}
```

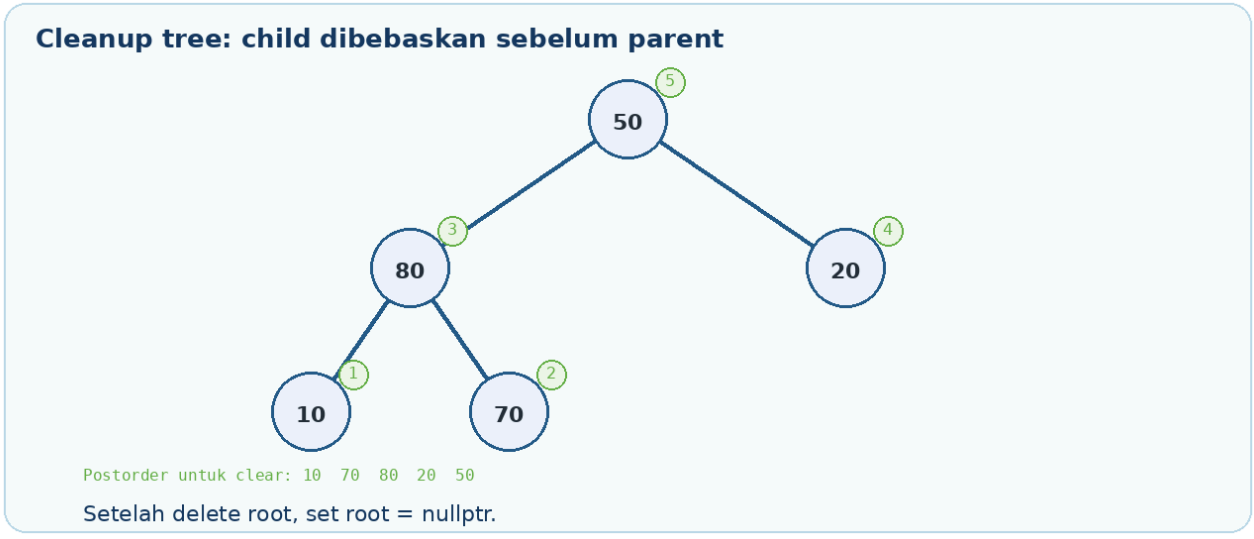
5.2 Membebaskan tree

Untuk membebaskan tree, child harus dibebaskan sebelum parent. Pola ini sama dengan postorder: Left - Right - Root.

```
void clear(Node*& root) {
    if (root == nullptr)
        return;

    clear(root->left);
    clear(root->right);

    delete root;
    root = nullptr;
}
```



Gambar 6. Cleanup tree mengikuti pola postorder: child sebelum parent.

6. Kegiatan Praktikum

Gunakan satu berkas bernama latihan_binary_tree.cpp. Kompilasi program setelah setiap perubahan.

6.1 Membentuk binary tree

Buat tree berikut secara manual menggunakan new. Tree sengaja tidak mengikuti aturan Binary Search Tree.

```
Node* root = new Node{50, nullptr, nullptr};
root->left = new Node{80, nullptr, nullptr};
root->right = new Node{20, nullptr, nullptr};
root->left->left = new Node{10, nullptr, nullptr};
root->left->right = new Node{70, nullptr, nullptr};
root->right->right = new Node{90, nullptr, nullptr};
root->left->right->left = new Node{30, nullptr, nullptr};
```

1. identifikasi root, parent, child, sibling, dan leaf pada tree;
2. pilih satu node dan tunjukkan subtree yang berakar pada node tersebut;
3. jelaskan mengapa left child tidak selalu mempunyai nilai yang lebih kecil; dan
4. periksa keadaan tree kosong dan tree satu node pada contoh terpisah.

6.2 Prediksi dan implementasi traversal

Sebelum menjalankan program, tuliskan prediksi hasil untuk tree pada Kegiatan 6.1:

```
Preorder  : _____________________________
Inorder   : _____________________________
Postorder : _____________________________
```

1. implementasikan preorder();
2. implementasikan inorder();
3. implementasikan postorder();
4. jalankan ketiganya; dan
5. bandingkan hasil aktual dengan prediksi.

6.3 Menghitung node dan cleanup

1. lengkapi fungsi jumlahNode();
2. pastikan hasilnya sesuai jumlah node pada gambar;
3. jalankan clear(root);
4. periksa bahwa root == nullptr; dan
5. panggil clear(root) sekali lagi dan pastikan program tetap aman.

7. Tugas Praktikum

Buat program Eksplorasi Binary Tree dan simpan dalam modul7_binary_tree.cpp. Gunakan binary tree non-BST yang diberikan pada tugas oleh asisten atau bentuk tree yang ditentukan pada modul.

Program harus dapat:

1. membentuk binary tree menggunakan node dinamis;

2. menampilkan preorder;
3. menampilkan inorder;
4. menampilkan postorder;
5. menghitung jumlah node;
6. membandingkan prediksi traversal dengan hasil program; dan
7. membebaskan seluruh node sebelum program berakhir.

7.1 Kerangka program

```
#include <iostream>

struct Node {
    int data;
    Node* left;
    Node* right;
};

void preorder(const Node* root);
void inorder(const Node* root);
void postorder(const Node* root);
int jumlahNode(const Node* root);
void clear(Node*& root);

int main() {
    Node* root = nullptr;

    // bentuk tree sesuai gambar tugas

    std::cout << "Preorder  : ";
    preorder(root);
    std::cout << '\n';

    std::cout << "Inorder   : ";
    inorder(root);
    std::cout << '\n';

    std::cout << "Postorder : ";
    postorder(root);
    std::cout << '\n';

    std::cout << "Jumlah node: " << jumlahNode(root) << '\n';

    clear(root);
    return 0;
}
```

7.2 Fungsi yang dilengkapi

Lengkapi fungsi traversal, jumlah node, dan clear setelah main. Jangan mengubah root atau hubungan tree pada fungsi traversal.

```
void preorder(const Node* root) {
    // lengkapi
```

```
}

void inorder(const Node* root) {
    // lengkapi
}

void postorder(const Node* root) {
    // lengkapi
}

int jumlahNode(const Node* root) {
    // lengkapi
    return 0;
}

void clear(Node*& root) {
    // lengkapi
}
```

8. Pengujian

Jalankan seluruh skenario berikut. Prediksi traversal harus ditulis sebelum hasil program dilihat.

No.	Skenario	Hasil yang diharapkan
1	Tree baru	root == nullptr
2	Tree satu node	left dan right bernilai nullptr
3	Tree beberapa node	Hubungan left/right sesuai gambar
4	Preorder	Urutan Root-Left-Right benar
5	Inorder	Urutan Left-Root-Right benar
6	Postorder	Urutan Left-Right-Root benar
7	Prediksi pada tree baru	Prediksi sama dengan output program
8	jumlahNode()	Jumlah sesuai banyak node
9	clear()	Seluruh node dibebaskan dan root == nullptr

8.1 Pengujian kondisi batas

Selain tree utama, uji tiga kondisi berikut:

- tree kosong: semua traversal tidak menghasilkan output dan tidak crash;
- satu node: preorder, inorder, dan postorder menghasilkan nilai yang sama; dan
- tree tidak seimbang: semua node tetap harus dikunjungi tepat satu kali.

8.2 Diagnosis kode

Setiap potongan berikut mengandung kesalahan. Tentukan kesalahan, akibat, dan perbaikannya.

A. Base case tidak ada

```
void preorder(const Node* root) {
    std::cout << root->data << ' ';
    preorder(root->left);
```

```
    preorder(root->right);
}
```

B. Inorder ditulis seperti postorder

```
void inorder(const Node* root) {
    if (root == nullptr) return;
    inorder(root->left);
    inorder(root->right);
    std::cout << root->data << ' ';
}
```

C. Subtree kanan tidak ditelusuri

```
void preorder(const Node* root) {
    if (root == nullptr) return;
    std::cout << root->data << ' ';
    preorder(root->left);
}
```

D. jumlahNode() lupa menghitung node saat ini

```
int jumlahNode(const Node* root) {
    if (root == nullptr) return 0;
    return jumlahNode(root->left)
         + jumlahNode(root->right);
}
```

E. Parent dihapus sebelum child

```
delete root;
clear(root->left);
clear(root->right);
```

F. root tidak di-null-kan setelah clear

```
clear(root->left);
clear(root->right);
delete root;
```

9. Pengayaan

Setelah seluruh kompetensi inti selesai, praktikan dapat mencoba:

- membuat fungsi jumlahLeaf() untuk menghitung leaf;
- membuat fungsi height() untuk menghitung tinggi tree; atau
- membentuk tree lain dan memprediksi ketiga traversal tanpa menjalankan program terlebih dahulu.

10. Jembatan ke Modul 8

Binary tree pada modul ini belum mensyaratkan urutan nilai tertentu. Pada Modul 8, binary tree diberi invariant pengurutan sehingga terbentuk Binary Search Tree. Invariant tersebut kemudian digunakan untuk melakukan search, insertion, dan deletion secara terarah.